



NLP Unit 5

Natural Language Processing (Jawaharlal Nehru Technological University, Hyderabad)



Scan to open on Studocu

UNIT -V

language model :

A language model is a probability distribution over sequence of words, Given such a sequence of length 'm', a language model assigns a probability P to the whole sequence. Language models generate probabilities by training of text corpora in one or many languages.

Language model form the backbone of the NLP.

They are a way of transforming qualitative information about text into quantitative information that machines can understand.

N-gram model :

N-gram are continuous sequences of words or symbols or tokens in a document. In technical terms, they can be defined as the neighbouring sequences of item. They come into play when we deal with text-data in NLP.

What are n-grams:

N-grams are continuous sequence of words or symbols or tokens in a document. In technical terms, they can be defined as the neighbouring sequence of items.

n-grams classifier:

'n' is just a variable that can have +ve integer values including 1, 2, 3 and so on.

→ 'n' basically refers to multiple.

n	Term
1	Unigram
2	Bigram
3	Trigram
n	n-gram

when $n=1$, it is said to be unigram

$n=2$, it is said to be a bigram

$n=3$, it is said to be a trigram

ex: Language model:

$$p(x, y) = p(x/y) p(y)$$

$$p(x, y, z) = p(x) p(y/x) p(z/x, y)$$

$$p(\text{she, is, dead}) = p(\text{she}) \\ p(\text{is/she}) \\ p(\text{dead/she is})$$

Markov Assumption:

I wish was a unicorn

$$\rightarrow p(\text{unicorn} / \text{I wish was a}) \approx p(\text{unicorn/a})$$

$$\text{or } p(\text{unicorn/was a})$$

ex: N-gram such as unigram

("This", "article", "is", "on", "NLP") or

bigram: ("This article", "article is", "is on", "on NLP")

Now, we will establish a relation on how to find the next word in the sentence.

$P(\text{NLP} / \text{This article is on})$

equation: $P(w_5 / w_1 w_2 w_3 w_4)$ or $P(w)$
 $= P(w_n / w_1 w_2 \dots w_n)$

Calculation: $P(A/B) = \frac{P(A,B)}{P(B)}$

$$P(A,B) = P(A/B) P(B)$$

Corpus:

$\langle s \rangle$ I am a human $\langle /s \rangle$

$\langle s \rangle$ I am not a stone $\langle /s \rangle$

$\langle s \rangle$ I I live in Mumbai $\langle /s \rangle$

→ check the probability of

$\langle s \rangle$ I am not $\langle /s \rangle$ using

→ (I I am not)

$$= P(I / \langle s \rangle) P(I / I) P(am / I) P(not / am) P(\langle /s \rangle / not)$$

$$= \frac{C(\langle s \rangle / I)}{C(\langle s \rangle)} \cdot \frac{C(I / I)}{C(I)} \cdot \frac{C(I / am)}{C(I)} \cdot \frac{C(am / not)}{C(am)}$$

$$\frac{C(not / \langle /s \rangle)}{C(not)}$$

$$= \frac{3}{3} \cdot \frac{1}{4} \cdot \frac{2}{4} \cdot \frac{1}{2} \cdot 0$$

$$= 0$$

→ Consider the following training data

<S> I am Jack </S>

<S> Jack I am </S>

<S> Jack I like </S>

<S> Jack I do like </S>

<S> do I like Jack </S>

Assume that we use a bigram.

language model based on above data.

and is most probable next word predicted by the model

1. <S> Jack 2. <S> Jack I do ...

3. <S> Jack I am Jack ...

4. <S> do I like ...

$$P(I / <S>) = \frac{C(<S> / I)}{C(<S>)} = 1/5$$

$$P(\text{Jack} / <S>) = \frac{C(<S> / \text{Jack})}{C(<S>)} = 3/5$$

$$P(\text{do} / <S>) = \frac{C(<S> / \text{do})}{C(<S>)} = 1/5$$

$$P(\text{am} / I) = \frac{C(I / \text{am})}{C(I)} = 2/5$$

$$P(\text{like} / I) = \frac{C(I / \text{like})}{C(I)} = 1/5$$

$$P(\text{do} / I) = \frac{C(I / \text{do})}{C(I)} = 2/5$$

$$P(</S> / \text{Jack}) = \frac{C(\text{Jack} / </S>)}{C(\text{Jack})} = 2/5$$

$$P(</S> / \text{like}) = \frac{C(\text{like} / </S>)}{C(\text{like})} = 2/3$$

$$P(</S> / \text{am}) = \frac{C(\text{am} / </S>)}{C(\text{am})} = 1/2$$

$$P(I | Jack) = C(Jack/I) / C(Jack) = 3/5$$

$$P(like | do) = C(do/like) / C(do) = 1/2$$

$$P(Jack | like) = C(like/Jack) / C(like) = 1/3$$

$$P(Jack | am) = C(am/Jack) / C(am) = 1/2$$

1) Jack I

2) Jack I & like

3) Jack I am Jack - -

4) <S> do I like <S>

Trigram model:

A Trigram model approximates the probability of a word given all the previous words by using only the conditional probability of the preceding words while trigram model looks two words into the past.

Ex: <S> sun is a <S>
 <S> is a kind <S>
 <S> a kind soul <S>
 <S> kind soul she <S>
 <S> soul she will <S>

$$\text{for: } p(x, y, z) = p(x) p(y) [P(x, y/z)] = P(x, y/z)$$

$$\rightarrow p(<S>, sun, is) = p(<S>, sun/is) = 1/2 = 0.5$$

$$p(sun, is, a) = p(sun, is/a) = 1/3 = 0.333$$

$$p(is, a, <S>) = p(is, a/<S>) = 2/5 = 0.4$$

$$p(<S>, is, a) = p(<S>, is/a) = 1/3 = 0.333$$

$$P(rs, a, kind) = P(rs, a / kind) = \frac{2}{3} = 0.6666$$

$$P(a, kind, \langle /s \rangle) = P(a, kind / \langle /s \rangle) = \frac{2}{5} = 0.4$$

$$P(\langle /s \rangle, a, kind) = P(\langle /s \rangle, a / kind) = \frac{1}{3} = 0.3333$$

$$P(a, kind, soul) = P(a, kind / soul) = \frac{2}{3} = 0.666$$

$$P(kind, soul, \langle /s \rangle) = P(kind, soul / \langle /s \rangle) = \frac{2}{5} = 0.4$$

$$P(\langle /s \rangle, kind, soul) = P(\langle /s \rangle, kind / soul) = \frac{1}{3} = 0.333$$

$$P(kind, soul, she) = P(kind, soul / she) = \frac{2}{2} = 1$$

$$P(soul, she, \langle /s \rangle) = P(soul, she / \langle /s \rangle) = \frac{2}{5} = 0.4$$

$$P(\langle /s \rangle, soul, she) = P(\langle /s \rangle, soul / she) = \frac{1}{2} = 0.5$$

$$P(soul, she, will) = P(soul, she / will) = \frac{2}{1} = 2$$

$$P(she, will, \langle /s \rangle) = P(she, will / \langle /s \rangle) = \frac{1}{5} = 0.2$$

$$P = 0.5 \times 0.3333 \times 0.4 \times 0.333 \times 0.6666 \times 0.4 \times 0.333 \times 0.666 \times \\ 0.4 \times 0.333 \times 1 \times 0.4 \times 0.5 \times 2 \times 0.5$$

$$P = 0.00001403821 //$$

$$\therefore \text{Total probability } P = 0.00001403821$$

Language Model Evaluation:

Before descriptive parameter estimation method further refinements of the basic n-gram modeling approach.

Let us consider the problem of judging the performance of a language model.

Unigram

This	is	Big	Data	AI	Book
------	----	-----	------	----	------

Bi-gram

This is	is Big	Big Data	Data AI	AI Book
---------	--------	----------	---------	---------

Tri-gram

This is Big	is Big Data	Big Data AI	Data AI Book
-------------	-------------	-------------	--------------

Text N-gram

Data 4-gram

great info 2-gram

Nice meeting you 3-gram

Nice to meet you 4-gram

• Evaluating the strings:

<sy I want to eat chinese food <1s>

No. of tokens = 8

No. of sentences = 1

No. of paragraphs = 0

No. of types = 1

→ Counting:	N-gram	Unigram	Bigram	Trigram
	Text probability	962	170	109

$$p = p(w_1, w_2, \dots, w_n)$$

$$\text{Bigram}(P) = \frac{C(w_n)}{C(w_1)}$$

$$\text{Trigram}(T) = C\left(\frac{(w_1, w_2)}{w_n}\right)$$

Parameter Estimation:

1. Maximum-Likelihood Estimation and Smoothing:

So far, we have discussed estimating the mean and variance of a distribution. Our methods have been some what and have more specifically. If it is not clear how we can estimate other parameters.

Now we would like to talk about a systematic way of parameter estimation.

Definition: The process of redistributing probability means search to some small & non zero value is called smoothing. The most common smoothing technique is back-off. It includes splitting n-grams into those counts exceed the threshold.

Ex:

- <S> I am a human <S>
- <S> I am not a stone <S>
- <S> I I live in Lahore <S>

$$\begin{aligned} \Rightarrow P(\text{I am a human}) &= P(I/ks) P(am/I) P(a/am) P(\frac{\text{human}}{a}) \\ &= 3/3 \cdot 2/4 \cdot 1/2 \cdot 1/2 \\ &= 0.125 \end{aligned}$$

→ Split vocabulary:

$$\begin{aligned} P(\text{I am human}) &= P(I/ks) P(am/I) P(\text{human}/am) \\ &= 3/3 \cdot 2/4 \cdot 1/2 = 0.25 \end{aligned}$$

General Bigram probability: $P(x/y) = C(xy)/C(y)$

→ Bigram probability with "Laplace smoothing":

$$P(x/y) = \frac{C(xy) + 1}{C(y) + v}$$

where v = distinct words

$$P(I/am) = \frac{2+1}{2+11} = 0.230$$

Trigram Training set:

denied the allegations

denied the reports

denied the claims

denied the request

→ probability using simple trigram model

Denied the - offer - ?

$$\Rightarrow P(\text{offer} | \text{denied the}) = \frac{\text{count}(\text{denied the offer})}{\text{count}(\text{denied the})} = 0/4 = 0$$

→ probability using "Laplace smoothing or add one smoothing"

$$p(\text{offer} | \text{denied the}) = \frac{\text{count}(\text{denied the offer}) + 1}{\text{count}(\text{denied the}) + v}$$

$$= \frac{0+1}{4+6} = \frac{1}{10} = 0.1$$

⇒ steal probability mass to generalize better

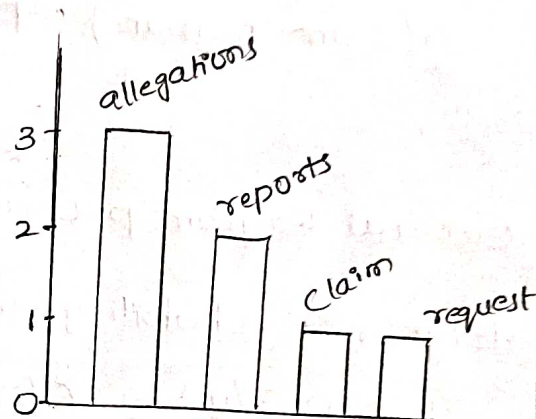
denied¹ the² allegation³ = 3

denied¹ the² reports³ = 2

denied¹ the² claim³ = 1

denied¹ the² request³ = 1

total : 7



⇒ Bigram Example given table:

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

→ Bigram counts for 8 of words ($v=1446$)

i want to eat chinese food lunch spend.

2533 927 2417 746 158 1093 341 278

⇒ what is the probability of the sentence "I want to"

$$\begin{aligned}
 p(\text{I want to}) &= p(\text{want/I}) * p(\text{want/want}) * p(\text{to/want}) \\
 &= \frac{c(\text{I want})}{c(\text{I})} * \frac{c(\text{want want})}{c(\text{want})} * \frac{c(\text{want to})}{c(\text{to})} \\
 &= \frac{827}{2533} * \frac{0}{927} * \frac{608}{2417} = 0 //
 \end{aligned}$$

→ using Laplace or add 1 smoothed bigram model

$$= \frac{827+1}{2533+1446} * \frac{0+1}{927+1446} * \frac{608+1}{2417+1446} = 1$$

Table after adding 1 in all values:

	I	want	to	eat	chinese	food	lunch	spend
I	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Bayesian Parameter Estimation:

Bayesian probability estimation is an alternative parameter estimation method where by the set of parameters of a model is itself viewed as a random variable

governed by a prior statistical distribution. given training sample is:

training set:

"he is he is ^{he is} going abroad is going to study in the field"

⇒ $p(\text{is going abroad to study}) = ?$

⇒ using bi-gram model:

$$\begin{aligned} &= p(\text{going}|\text{is}) * p(\text{abroad}|\text{going}) * p(\text{to}|\text{abroad}) * p(\text{study}|\text{to}) \\ &= \frac{\text{count}(\text{is going})}{\text{count}(\text{is})} * \frac{\text{count}(\text{going abroad})}{\text{count}(\text{going})} * \frac{\text{count}(\text{abroad to})}{\text{count}(\text{abroad})} * \frac{\text{count}(\text{to study})}{\text{count}(\text{to})} \end{aligned}$$

$$= \frac{2}{4} * \frac{1}{2} * \frac{0}{1} * \frac{1}{1} = 0 //$$

⇒ using Laplace smoothing

Distinct vocabulary = $v = 9$

⇒ $p(\text{is going abroad to study})$

$$= \left(\frac{2+1}{4+9} \right) * \left(\frac{1+1}{2+9} \right) * \left(\frac{0+1}{1+9} \right) * \left(\frac{1+1}{1+9} \right)$$

$$= \frac{3}{13} * \frac{2}{11} * \frac{1}{10} * \frac{2}{10} = 0.000839$$

⇒ using Bayesian:

"he is he is he is going abroad is going to study in the field"

⇒ $p(\text{is going abroad to study}) = ?$

total distinct words = $v = 9$

(is going, going abroad, abroad to, to study)

pairs of distinct values = 4 //

pairs of distinct values = 9

- using Laplace smoothing add $v=4$

$$= \left(\frac{2+1}{4+4} \right) * \left(\frac{1+1}{2+4} \right) * \left(\frac{0+1}{1+4} \right) * \left(\frac{1+1}{1+4} \right)$$

$$= \frac{3}{8} * \frac{2}{6} * \frac{1}{5} * \frac{2}{5} = \frac{12}{1200} = 0.01 //$$

Large-scale language model:

These language models (LSM) estimate the relative likelihood of different phrases and are useful in many different NLP applications.

Ex: They have been used in twitter bots for robot accounts to for own sentences.

• Defining language models: The goal of probabilistic language modeling is to calculate the prob of a sentence of sequence of words.

$$P(w) = P(w_1 w_2 w_3 \dots w_n)$$

→ And can be to find the probability of the next word in the sequence.

$$P(w_5 / w_1 w_2 w_3 w_4)$$

Conditional probability:

Let A, B be two events with $P(B) \neq 0$

Conditional probability of A given B is:

$$P(A/B) = \frac{P(A, B)}{P(B)}$$

Chain Rule:

In general cases, the formula is as follows:

$$P(x_1, x_2, \dots, x_n) = P(x_1) P(x_2/x_1) \dots P(x_n/x_{n-1})$$

ex: $P(\text{"this water is so transparent"})$

$$\rightarrow P(\text{This}) * P\left(\frac{\text{water}}{\text{this}}\right) * P\left(\frac{\text{is}}{\text{this water}}\right) *$$

$$P\left(\frac{\text{so}}{\text{this water is}}\right) * P\left(\frac{\text{transparent}}{\text{this water is so}}\right)$$

Markov Property:

A stochastic process has a Markov Property if the conditional probability distribution of future states of the process depends only on the present state, not on sequence of events that preceded it. In other words, probability of next word can be estimated given only the previous k no. of words.

$\rightarrow$ if $k=1$;

$$P(\text{transparent} / \text{this water is so}) \approx P(\text{transparent} / \text{so})$$

or if $k=2$;

$$P(\text{transparent} / \text{this water is so}) \approx P(\text{transparent} / \text{is so})$$

General eq'n for all the Markov Assumption

$$\text{if } k=i; \text{ then } P(w_i / w_1 w_2 \dots w_{i-1})$$

Types of Language Models:

1) Class-based language model:

Class-based N-gram model represent a common meaning for improving the robustness of language modeling by providing adequate representation of words or sequences using Markov assumption not existing or under represented in training or corpus.

$$P(w_i / w_{i-1}) = P(w_i / c_i) P(w_i / c_{i-1})$$

Ex: $P(I \text{ saw a cat on a mat})$

$$\Rightarrow P(\text{saw} / I)$$

$$P(a / I \text{ saw})$$

$$P(\text{cat} / I \text{ saw } a)$$

$$P(\text{on} / I \text{ saw a cat})$$

$$P(a / I \text{ saw a cat on})$$

$$P(\text{mat} / I \text{ saw a cat on } a)$$

→ After class-based model eliminate or split before word in the denominator in the bigram

$$P(I \text{ saw a cat on a mat})$$

$$P(I) \rightarrow P(I)$$

$$P(\text{saw} / I) \rightarrow P(\text{saw} / I)$$

$$P(a / I \text{ saw}) \rightarrow P(a / I \text{ saw})$$

$$P(\text{cat} / I \text{ saw } a) \rightarrow P(\text{cat} / \text{saw } a)$$

$$P(\text{on} / I \text{ saw a cat}) \rightarrow P(\text{on} / a \text{ cat})$$

$$P(a / I \text{ saw a cat on}) \rightarrow P(a / \text{cat on})$$

$P(\text{mat} / \neq \text{sw} \text{ \& \& cat on a}) \rightarrow P(\text{mat} / \text{on a})$

Estimating probability using sentence simplify bigram

using class-based model:

$\langle s \rangle$ I am sam $\langle /s \rangle$

$\langle s \rangle$ I am I am $\langle /s \rangle$

$\langle s \rangle$ I fly $\langle /s \rangle$

distinct words: I, am, sam, I, fly

Bigram: $\langle s \rangle$ and $\langle /s \rangle$ are also tokens.

There are $6(u+2)$ tokens and $6 \times 6 = 36$ bigrams.

$$P(I / \langle s \rangle) = 2/3 \quad P(\frac{\text{sam}}{\langle s \rangle}) = 1/3 \quad P(\frac{\text{am}}{\langle s \rangle}) = P(\frac{\text{fly}}{\langle s \rangle}) = 0 \quad P(\frac{\langle s \rangle}{\langle s \rangle}) = 0$$
$$P(\langle /s \rangle / \langle s \rangle) = 0$$

$$P(I / I) = 0 \quad P(\frac{\text{sam}}{I}) = P(\frac{\text{am}}{I}) = P(\frac{\text{fly}}{I}) = P(\frac{\langle s \rangle}{\langle s \rangle I}) = P(\frac{\langle /s \rangle}{I}) =$$

$$P(I / \text{am}) = 0 \quad P(\frac{\text{sam}}{\text{am}}) = P(\frac{\text{am}}{\text{am}}) = P(\frac{\text{fly}}{\text{am}}) = P(\frac{\langle s \rangle}{\text{am}}) = P(\frac{\langle /s \rangle}{\text{am}}) =$$

$$P(I / \text{sam}) = 1/2 \quad P(\frac{\text{sam}}{\text{sam}}) = 0 \quad P(\frac{\text{am}}{\text{sam}}) = 0 \quad P(\frac{\text{fly}}{\text{sam}}) = P(\frac{\langle s \rangle}{\text{sam}}) = P(\frac{\langle /s \rangle}{\text{sam}}) =$$

$$P(I / \text{fly}) = 0 \quad P(\frac{\text{sam}}{\text{fly}}) = P(\frac{\text{am}}{\text{fly}}) = P(\frac{\text{fly}}{\text{fly}}) = 0 \quad P(\frac{\langle s \rangle}{\text{fly}}) = P(\frac{\langle /s \rangle}{\text{fly}}) =$$

$$P(I / \langle /s \rangle) = P(\frac{\text{sam}}{\langle /s \rangle}) = P(\frac{\text{am}}{\langle /s \rangle}) = P(\frac{\text{fly}}{\langle /s \rangle}) = P(\frac{\langle s \rangle}{\langle /s \rangle}) = P(\frac{\langle /s \rangle}{\langle /s \rangle}) =$$

Trigram are $6 \times 6 \times 6 = 216$

Compute the probability of sentence.

I want English food

$P(\text{I want English food})$

② Variable-length language model:

Since probabilities are less than or equal to 1, the more probabilities we multiply together, the smaller the product becomes. multiply enough n-grams together result in numerical underflow.

By using log probabilities instead of raw probabilities, we get numbers that are not as small.

Adding in log space is equivalent to multiplying in linear space. so we combine log.

→ adding is faster than multiplying the result of doing all computing and storage in log space is that, we only need to convert back into prob if we need to report them at the end.

$$\log(P_1 P_2 P_3 P_4) = \log P_1 + \log P_2 + \log P_3 + \log P_4$$

$$P_1 * P_2 * P_3 * P_4 = \exp(\log P_1 + \log P_2 + \log P_3 + \log P_4)$$

③ Discriminative language model:

Standard n-gram models is a general model for assigning a probability to a given word sequence 'w'. However, in practical applications like machine translation or speech recognition, the task of a language model is often to separate good sentence hypotheses from bad sentence hypotheses. such that word strings of widely differing quality receive maximally.

Distinct probability estimates Recent attempts at such discriminative language model.

Ex:

Given data:

Jane saw will

Mary saw Jane

→ Collect data with pos tagging

Jane saw will Mary saw Jane
N V N N V N

Create lookup table:

	N	V
Mary	1	0
saw	0	2
Jane	2	0
will	1	0

→ three training data set:

^NMary ^Mwill ^Vsee ^NJane

^Nwill ^Mwill ^Vsee ^NMary

^NJane ^Mwill ^Vsee ^Nwill

Create lookup table:

	N	V	M
Mary	2	0	0
will	2	0	3
see	0	3	0
Jane	2	0	0

Bigram: N-followed by M

M followed by V

V followed by N

• Bigram lookup table:

	N-M	M-V	V-N
Mary - will	1	0	0
will - see	0	3	0
see - Jane	0	0	1
will - will	1	0	0
see - marry	0	0	1
Jane - will	1	0	0
see - will	0	0	1

Let us an example proposed and appropriate tag sequence for a sentence.

"John can see will"

↓ ↓ ↓ ↓
N modal verb Noun

first noun word after comes modal and a modal after a verb, verb after noun.

→ "Tied will spot will"

↓ ↓ ↓ ↓
N m v N

tagged noun, modal, verb, noun. These probabilities are emission prob.

ex②: mary Jane can see will
N N M V N

spot will see mary
N M V N

will Jane spot mary
M N V N

mary will pat sat
N M V N

• Create lookup table

	N	V	M
Mary	4	0	0
Jane	2	0	0
can	0	0	1
see	0	2	0
will	1	0	3
spot	1	1	0
pat	0	1	0
sat	1	0	0

→ Identify Noun, Model (M), verb (V). Let's divide each word appeared by the total no. of every pos in the set of sentence.

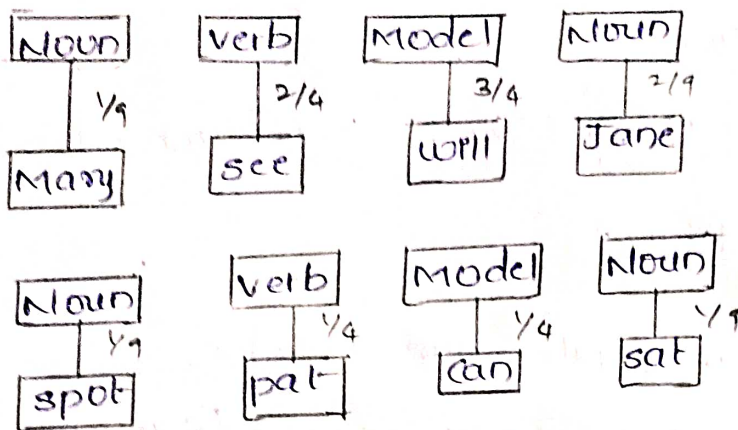
→ No. of N=9, No. of V=4, No. of M=4

	N	V	M
Mary	4/9	0/4	0
Jane	2/9	0	0
can	0	0	1/4
see	0	2/4	0
will	1/9	0	3/4
spot	1/9	1/4	0
pat	0	1/4	0
sat	1/9	0	0

→ Mary: $N = 4/9$, $V = 0$, $M = 0$

Jane: $N = 2/9$, $V = 0$, $M = 0$

⋮



$$\rightarrow \text{total probability} = \frac{1}{9} \cdot \frac{2}{4} \cdot \frac{3}{4} \cdot \frac{2}{9} \cdot \frac{1}{9} \cdot \frac{1}{4} \cdot \frac{1}{4} \cdot \frac{1}{9}$$

$$= 0.00694$$

④ Syntax-Based Language Model:

A well known drawback of n-gram language model is that they take account of similar words in the history. That fall outside the limited window of the directly preceding (n-1) words. However Natural language exhibits many types of long distance dependency the choice of the current word is dependent on the words that are relatively removed in terms of sequence position.

our goal is to compute the probability of sentence or sequence of words $w = (w_1, w_2, \dots, w_n)$

$$P(w) = P(w_1, w_2, \dots, w_n)$$

$$= P(w_n / w_1 w_2 w_3 \dots w_{n-1})$$

$\rightarrow$ And compute probability of a sequence and computing the Conditional probability of a word given previous words

Conditional probability: $P(B/A) = \frac{P(A, B)}{P(A)}$

$$\rightarrow P(A, B) = P(A)P(B/A)$$

$$\rightarrow P(A, B, C, D) = P(A)P(B/A)P(C/A, B)P(D/A, B, C)$$

$$\text{unigram: } P(w_1^n) \approx \prod_{k=1}^n P(w_k)$$

$$\text{Ex: } P(\langle s \rangle \text{ the man from jupiter came } \langle s \rangle) \approx \\ P(\text{the})P(\text{man})P(\text{from})P(\text{jupiter})P(\text{came})$$

$$\text{Bigram: } P(w_1^n) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

$$\text{Ex: } P(\langle s \rangle \text{ the man from jupiter came } \langle s \rangle) \approx \\ P(\text{the} / \langle s \rangle)P(\text{man} / \text{the})P(\text{from} / \text{man})P(\text{jupiter} / \text{from})P(\text{came} / \text{jupiter}) \\ \cdot P(\langle s \rangle / \text{jupiter came})$$

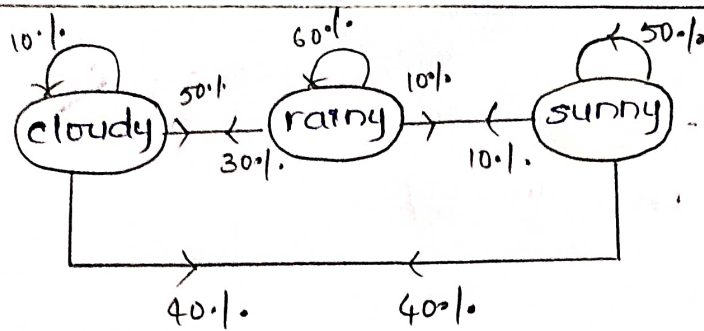
$$\text{Trigram: } P(w_1^n) \approx \prod_{k=1}^n P(w_k | w_{k-1} w_{k-2})$$

$$\text{Ex: } P(\langle s \rangle \text{ the man from jupiter came } \langle s \rangle) \approx \\ = P(\text{man} / \langle s \rangle \text{ the})P(\text{from} / \text{the man})P(\text{jupiter} / \text{man from})P(\text{came} / \text{from jupiter}) \\ \cdot P(\langle s \rangle / \text{jupiter came})$$

⑤ Max Ent language model:

This model combined with Max-likelihood-based. prob estimation for language model is that the constraints imposed by estimates solely derived from the training data are too strong.

Ex: The diagram below represents three states representing weather of the day. (cloudy, rainy, and sunny).



→ There are different states such as cloudy, rainy, and sunny. Following represents the transition probabilities based on above diagram.

- If sunny today, then tomorrow:

$$p(\text{sunny}) = 50\%$$

$$p(\text{rainy}) = 10\%$$

$$p(\text{cloudy}) = 40\%$$

- If rainy today, then tomorrow:

$$p(\text{rainy}) = 60\%$$

$$p(\text{sunny}) = 10\%$$

$$p(\text{cloudy}) = 30\%$$

- If cloudy today, then tomorrow:

$$p(\text{cloudy}) = 10\%$$

$$p(\text{rainy}) = 50\%$$

$$p(\text{sunny}) = 40\%$$

→ cloudy(wednesday) sunny(tuesday)

probability to cloudy(wednesday)

→ sunny - sunny(Tuesday) - cloudy(wed)

the prob to a cloudy on wednesday

$$= 0.5 * 0.4 = 0.2$$

→ sunny - rainy (tuesday) - cloudy (wed)

the prob to a cloudy on wednesday

$$= 0.1 * 0.3 = 0.03$$

→ sunny - cloudy (tuesday) - cloudy (wed)

the prob to a cloudy on wednesday

$$\Rightarrow 0.1 * 0.4 = 0.04$$

→ total prob to a cloudy on wednesday = $0.2 + 0.03 + 0.04 = 0.27$

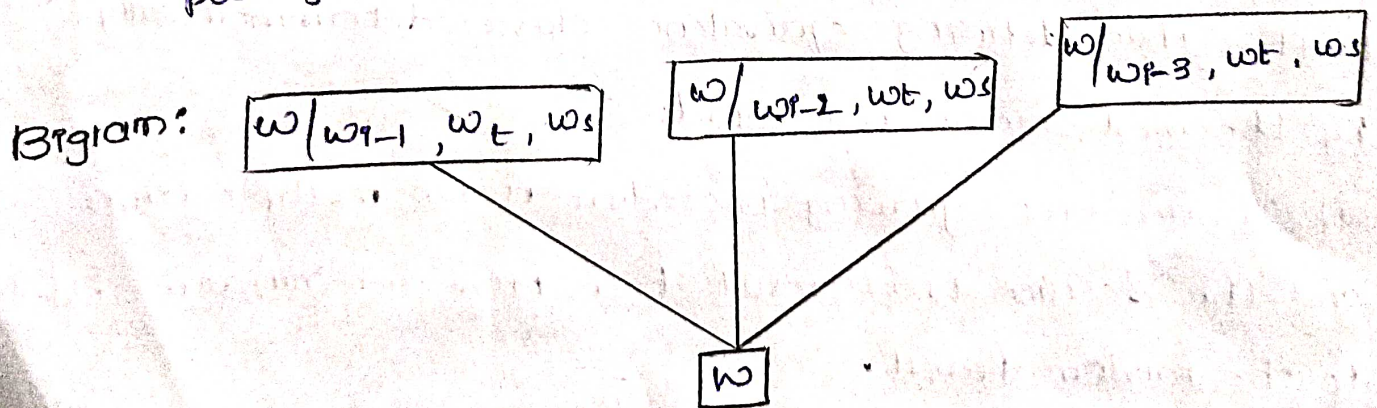
⑥ Factored language models:

The factored language model (FLM) approach [62, 63] builds on the observations that the prediction of words is dependent on the surface form of the preceding words and that better generalizations can be made when additional information, such as the word's pos or morphological class, is taken into account. In particular, word n-gram counts might be insufficient to robustly estimate the probability of word w_i given word w_{i-1} , but if we know that word w_{i-1} is of a particular class, say a determiner, we might be able to obtain a good probability estimate for $p(w_i / \text{determiner})$.

This is reminiscent of the class-based models described previously; however, in an FLM, many such class-based estimates are combined and structured hierarchically through the use of a generalized backoff strategy.

- FLMs assume a factored word representation, where words are considered feature vectors rather than individual surface forms; that is, $w \equiv f_1:k$.
- It is an extension of conventional language model.
- The probabilistic model provides where the prediction of a factor 'f' is based on 'n' parents ($f_1 \rightarrow f_n$).
- If w represents a word and t represents POS tag expression is $P(w/w_{i-1} w_{i-2} \dots w_{i-n})$.
- FLM used to allow stemming the word and provides POS tagging like morphological information.

Ex: word: stock prices are rising
 stem: stock price is rise
 pos tag: N N-plural v-plural v-part.

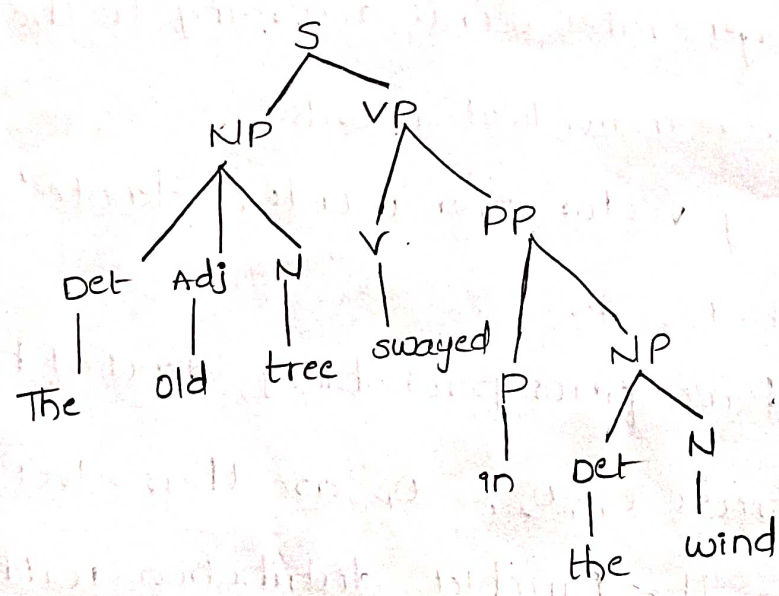


⑦ Other Tree-Based Language Models:

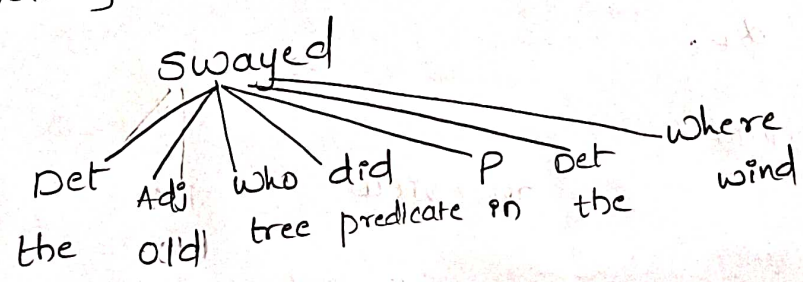
These model can be used for regular expression using CFG rules to generate prediction applying pos tagging to classify predicting categorization values.

- Random forest language (RFLMs) Models, (produced) proposed by Xu and Jelinek, model all word histories seen in the training data as a collection of randomly grown decision trees (a random forest).
- Nodes in the decision trees are associated with sets of histories, with the root node containing all histories.
- Trees are grown by dividing the set of histories into two subsets according to the word identity at a particular position in the history.
- Out of a number of possible splits chosen to undergo the log-likelihood test is selected randomly as well.
- After the growing process has stopped, each leaf node in a decision tree can be viewed as a cluster of similar word histories forming an equivalence class.
- Rather than defining equivalence classes deterministically by the most recent $n-1$ words.
- The decision tree-growing procedure is run multiple times, and the decision trees result been from each run are added to the random forest.
- Supposing that M decision trees have been obtained, the RFLM probabilities are computed as averages of the individual decision tree probabilities.

Ex: The old tree swayed in the wind
 who ↓ predicate
 ↓ did where.



dependency tree:



⑧ Bayesian Topic-Based Language Models:

- A significant recent trend in statistical language modeling is Bayesian modeling of latent topic structure in documents.
- It is a probabilistic algorithm based on applying bayes theorem with Naïves assumption of conditional independent between every pair of feature. Given example training data.
- One of the first models of this type was the latent Dirichlet allocation model proposed by Blei, Ng, & Jordan.

- The LDA model assumes that a document is composed of K topics, denoted as $z_1, \dots, z_K$.
- Each topic generates words according to the topic-specific distribution over individual words.
- The probability vector over words is denoted by ϕ_k for each topic $k=1, \dots, K$.
- Each topic has a prior probability, denoted by θ_k .
- the topic priors $\theta_1, \theta_2, \dots, \theta_K$ are themselves distributed according to the Dirichlet distribution with hyper-parameters $\alpha_1, \dots, \alpha_K$.

Ex: Given training data

Text	Review
I liked the movie	positive
It's a good movies nice story	positive
Nice songs but badly boring ending	Negative
Hero acting is bad overall nice	positive
movie heroine acting is good	
Bad boring movie	Negative

→ Classify the text and calculate the probability of $p(\text{"overall liked the movie"})$. is.

$$P(+ve) = 3/5, P(-ve) = 2/5$$

$$\rightarrow P(\text{overall liked the movie}) = P(\text{overall}) P(\text{liked}) P(\text{the}) P(\text{movie})$$

$$\rightarrow \text{Calculate } P(\text{overall liked the movie} | +ve) =$$

$$P\left(\frac{\text{overall}}{+ve}\right) P\left(\frac{\text{liked}}{+ve}\right) P\left(\frac{\text{the}}{+ve}\right) P\left(\frac{\text{movie}}{+ve}\right)$$

$$= \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{4}{3} = 0.04938 //$$

$$\rightarrow \text{Calculate } P(\text{overall liked the movie} | -ve) =$$

$$P\left(\frac{\text{overall}}{-ve}\right) P\left(\frac{\text{liked}}{-ve}\right) P\left(\frac{\text{the}}{-ve}\right) P\left(\frac{\text{movie}}{-ve}\right)$$

$$= \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{4}{2} = \frac{1}{4} = 0.25 //$$

⑨ Neural Network Language Models:

Neural Network language models (NNLMs) take a different approach: discrete word sequences are first mapped into a continuous representation, and n-gram probabilities are then estimated within this continuous space.

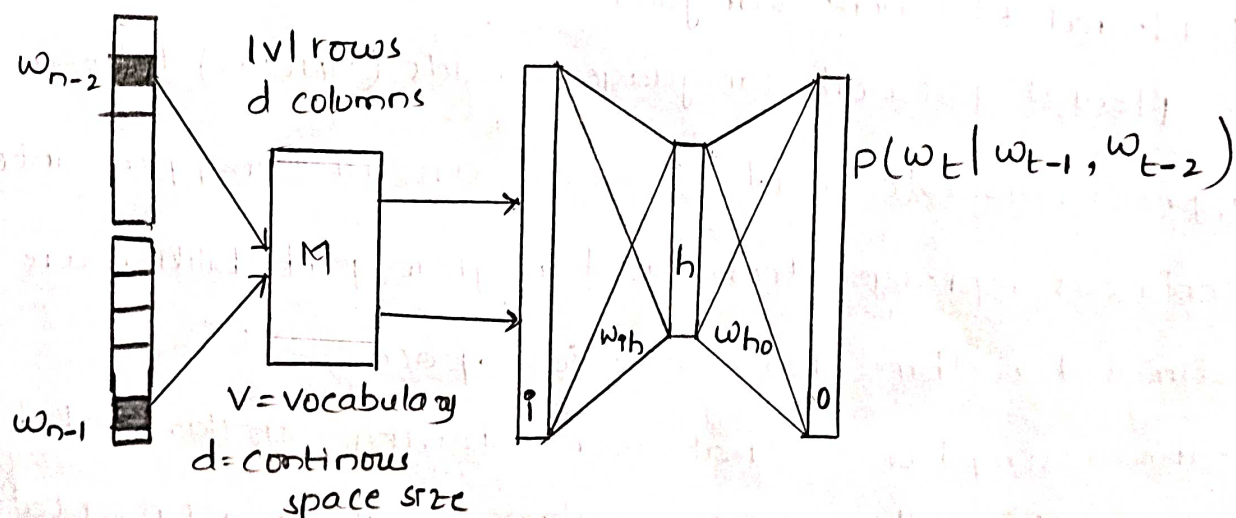
→ The assumption is that words having similar distributional properties will receive similar continuous representations, which will in turn result in smoother probability estimates.

→ The Neural Network is typically a multilayer perceptron with an input layer, projection layer, a hidden layer, and an output layer of nodes.

→ A graphical representation of the architecture of an NNLM is shown below.

- Adjacent layers are fully interconnected by weights
- For a vocabulary of V words, the input consists of a Concatenation of $n-1$ V -dimensional binary feature vectors representing the history of $n-1$ words (eg: The first two words in a trigram).
- The projection layer 'i' of a given fixed dimensionality d encodes the shared continuous representation of the words, which is learned during training.
- The hidden layer 'h' also has a fixed no. of J nodes.

$$h_j = \tanh \left(\sum_{k=1}^d w_{jk}^h i_k + b_j^h \right) \quad \forall j, 1=1, \dots, J$$



Language - Specific Modeling problems:

- The majority of language modeling research has focused on the English language.
- However speech and language processing technology has been ported to a range of other language.

→ some of which have highlighted problems with the standard n-gram modeling approach and have necessitated modifications to the traditional language modeling framework. In this section, we look at three types of language-specific problems: morphological complexity, lack of word segmentation and spoken versus written languages.

① Language Modeling for Morphologically Rich languages:

- A morphologically rich language is characterized by a larger number of different-unique word forms in relation to the no. of word tokens in a text that is due to productive morphological processes in the language.
- A morpheme is the smallest meaning-bearing unit in a language.
- Morphemes can be either free (i.e., they can occur on their own) or they are bound.
- Morphological processes include compounding (forming a new word out of 2-independently existing free morphemes), derivation (combination of a free and a bound morpheme to signal a particular grammatical feature).
- Many languages have rich inflectional paradigms.
- Morphological complexity causes problems for language modeling due to the high ratio of types of tokens, which

results in a lack of training data: many n-grams in the test data are not seen at all in the training data or are not observed frequently enough, leading to unreliable probability estimates.

Table : MSA inflectional paradigms for present tense verb forms and possessive pronouns (affixes are separated from the word stem by hyphens)

Word	Meaning	Word	Meaning
'a-skun(u)	I live	kitaab-iy	my book
ta-skun(u)	you (MASC) live	kitaabu-ka	your (MASC) book
ta-skun-iy-na	you (FEM) live	kitaabu-ki	your (FEM) book
ya-skun(u)	he lives	kitaabu-hu	his book
ta-skun(u)	she lives	kitaabu-haa	her book
na-skun(u)	we live	kitaabu-na	our book
ta-skun-uwna	you (MASC-PL) live	kitaabu-kum	your book
ya-skun-uwna	they live	kitaabu-hum	their book

⑫ Selection of subword units:

- The identification of subword units can be performed in a data-driven, unsupervised way
- It can be based on linguistic information (e.g. morphological analyzer), or it can be based on combination of both.
- Linguistically based methods typically involve a handcrafted morphological analyzer tool, such as the Buckwalter Morphological Analyzer, which analyzes each word form into its

morphological components.

- Data-driven approaches may incorporate varying degrees of information about the specific language.
- Some approaches are intended to discover units corresponding to linguistically defined morphemes.
- Automatic algorithms for identifying linguistic morphemes are as old as Zelling Harris's approach of estimating the perplexity of different letters.
- Morfessor package is one example. It attempts to derive a morpheme inventory M from a corpus c by maximizing its posterior probability:

$$M = \underset{M}{\operatorname{argmax}} P(M/c) = P(c/M) P(M)$$

(iii) Modeling with Morphological Categories:

- Most of the work on language modeling with subword units has focused on linear decomposition of words, primarily for agglutinative languages.
- As a result subword units are most frequently used in a standard n -gram model.
- One problem is that the n -gram context needs to be increased in order to be able to model interword dependencies in addition to dependencies b/w subword units.
- This in turn places demands on the amount of training data needed.

- The language model assigns probabilities to sequences of entire words, but does so by taking into account the constraints provided by morpheme-based feature functions.
- This model was shown to provide slightly better results (0.3% absolute word-error rate reduction on Broadcast News recognition task) than a discriminative language model using word-based features.
- The language model predicts the probability of the word string and the associated tree jointly, using MaxEnt probability estimation.
- The model was evaluated on an English-to-Arabic translation task improved the points over a word trigram model.

(iv) Languages without word segmentation:

- Although agglutination produces a large number of long and complex word forms in many languages, other languages do not possess any explicit segmentation of character strings into words at all.
- In languages such as Chinese and Japanese, sentences are written as sequence of characters delimited by punctuation signs but without any intervening whitespaces.
- Rather than trying to match linguistically defined words, it is also possible to optimize segmentation directly for language model performance.

→ Sproat et al. showed that the dictionary used for Chinese word segmentation significantly influences the perplexity of a bigram language model trained on the segmented text and that it is possible to iteratively optimize the dictionary by merging frequent word co-occurrences.

such that the perplexity is reduced at each iteration.

Note: such approaches are similar to the Data-driven algorithm for deriving morpheme-like subword units.

→ Another example of a data-driven approach, in this case for Japanese, uses chunks of characters for language modeling.

→ chunks are derived from training data by selecting the highest-frequency n-grams and patterns.

⑤ Spoken versus Written Languages:

→ Statistical language modeling crucially relies on large amounts of written text data, and significant trend within language modeling research is the development of methods for scaling current language modeling techniques to ever-larger databases. However, many of the world's 6,900 languages are spoken languages, that is, languages without a writing system.

They are either indigenous languages without a literary

tradition.

- For example, with the many dialects of Arabic which are used in everyday conversation but are almost never found in written form.
- Other languages may be spoken as well as written but may not have a standardized orthography.
- Only way of obtaining language model training data is to manually transcribe the language or dialect.
- This is a costly and time-consuming process because it involves
 - (i) the development of a writing standard
 - (ii) training native speakers to use the writing system accurately
 - (iii) the actual transcription effort.

Multilingual and Crosslingual Language Modeling

(i) Multilingual Language Modeling

up to this point we have discussed problems that arise when tailoring statistical language models to a particular language or language type, such as agglutinative languages or languages without word segmentation.

→ The tacit assumption has been that the resulting language model is used in an application where only the language of interest is encountered.

→ Here speakers may use several languages or dialects

side by side, often within the same utterance.

- The phenomenon of code switching exists in a variety of bilingual and multilingual communities or where diglossia exists, such as where a formal std language is used in addition to colloquial or dialectal varieties.

② Crosslingual language Modeling:

Another question that might be asked is whether data in one language can be leveraged to improve a language model for a different language.

- The most straightforward way of applying this idea is to automatically translate the foreign-language text into the desired language and to use the translated text as additional language training data.
- A unigram extracted from the translated text was interpolated with a trigram baseline language model.
- The use of machine translation technology for processing out-of-language data is likely to fail when the desired language does not have sufficient data to train a machine translation system.
- A drawback of the previous approaches is that the quality of the resulting model is heavily dependent on the translation accuracy.

for:
$$P_{En}(w) = \sum_k \phi_k^{En}(w) \theta_k^{ch}$$

• Assuming that the source language is Chinese (ch) and target language is English (En).

• $P_{En}(w)$ is the marginal word probability prob distribution of English.

• $\phi_k(w)$ is the probability assigned to word 'w'.

θ_k is the prior for the k^{th} topic.

→ target-language marginals are then incorporated into the target-language model as follows:

$$P_{target}(w/h) \propto \left(\frac{P_{bLSA}(w)}{P_{base}(w)} \right)^{\beta} P_{base}(w/h)$$

• where P_{bLSA} is the adapted probability

P_{base} is the baseline probability

→ this approach enforces a one-to-one topic correspondence across languages.